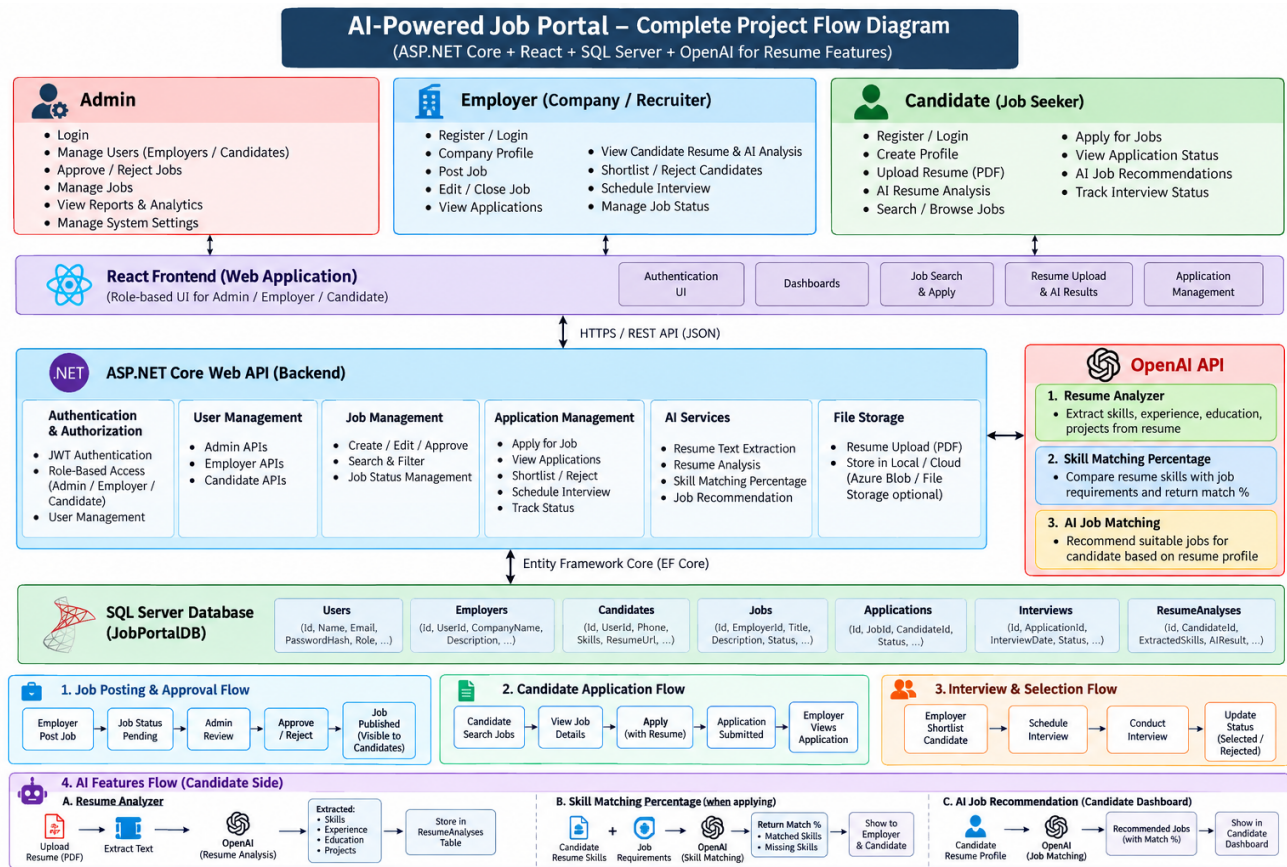


AI-Powered Job Portal

Complete Project Overview — React + ASP.NET Core Web API + SQL Server + OpenAI



1. Project Objective

Build a role-based job recruitment platform where employers can publish and manage jobs, candidates can search and apply, and administrators can control the platform. AI is intentionally limited to three useful candidate-side capabilities: resume analysis, skill matching percentage, and AI-based job matching/recommendations.

2. Technology Stack

Layer	Technology
Frontend	React.js + TypeScript + React Router + Axios + Tailwind CSS
Backend	ASP.NET Core Web API / C#
ORM	Entity Framework Core
Database	Microsoft SQL Server
Authentication	JWT Authentication
Authorization	Role-Based Authorization — Admin / Employer / Candidate
AI	OpenAI API — only for resume analysis, skill matching and job matching
API Testing	Swagger / OpenAPI + Postman
Version Control	Git + GitHub

3. User Roles

Role	Purpose	Main Functions
Admin	Platform administrator	Manage users, employers, candidates and jobs; approve/reject jobs; reports
Employer	Company / HR / Recruiter	Company profile, post/edit/close jobs, view applications, shortlist/reject candidates, schedu
Candidate	Job seeker	Create profile, upload resume, search jobs, apply, track applications/interviews, use AI ma

4. Complete Business Workflow

Employer: Register/Login → Create company profile → Post job → Job remains Pending → Admin reviews → Approve → Job becomes visible → Candidates apply → Employer views applications → Shortlist/Reject → Schedule interview → Interview → Selected/Rejected.

Candidate: Register/Login → Create profile → Upload PDF resume → AI resume analysis → Search/browse approved jobs → View AI match percentage → Apply → Track application → Attend scheduled interview → View final status.

Admin: Login → Manage users → Manage employers/candidates → Review jobs → Approve/Reject jobs → Manage jobs → View reports/analytics.

5. AI Scope — Only 3 Features

A. Resume Analyzer

Candidate uploads a PDF resume. The backend extracts readable text and sends the relevant text to OpenAI. The returned structured result can contain skills, experience, education and projects. The result is stored in SQL Server so the candidate does not need to analyze the same resume repeatedly.

B. Skill Matching Percentage

When a candidate views/applies for a job, the system compares the candidate's extracted resume skills with the job's required skills. OpenAI returns a match percentage plus matched and missing skills. The percentage is an assistance signal, not an automatic hiring decision.

C. AI Job Matching

The system compares the candidate's profile/resume with approved jobs and returns suitable job recommendations, including a match percentage and the relevant matching skills.

6. SQL Server Database Design

Table	Important Columns	Purpose
Users	Id, Name, Email, PasswordHash, Role, CreatedAt	Authentication and role
Employers	Id, UserId, CompanyName, Description, Location, Website	Employer/company profile
Candidates	Id, UserId, Phone, Location, Skills, Experience, ResumeUrl	Candidate profile
Jobs	Id, EmployerId, Title, Description, Requirements, Location, SalaryMin, SalaryMax, Status	Job listings
Applications	Id, JobId, CandidateId, CoverLetter, Status, AppliedAt	Candidate applications
Interviews	Id, ApplicationId, InterviewDate, Type, MeetingLink, Status, Feedback	Interview process
ResumeAnalyses	Id, CandidateId, ExtractedSkills, Experience, Education, AIResult	Stored AI resume analysis

7. Entity Relationships

Users 1 ■■ 1 Employer
Users 1 ■■ 1 Candidate
Employer 1 ■■ N Jobs
Candidate 1 ■■ N Applications
Job 1 ■■ N Applications
Application 1 ■■ N Interviews
Candidate 1 ■■ N ResumeAnalyses

8. Backend API Modules

Module	Example Endpoints
--------	-------------------

Auth	POST /api/auth/register, POST /api/auth/login
Jobs	GET /api/jobs, POST /api/jobs, PUT /api/jobs/{id}, DELETE /api/jobs/{id}
Applications	POST /api/jobs/{id}/apply, GET /api/employer/applications, PUT /api/applications/{id}/status
Interviews	POST /api/interviews, GET /api/interviews
Admin	GET/PUT user and job management endpoints
AI	POST /api/ai/analyze-resume, POST /api/ai/skill-match, GET/POST /api/ai/job-matches

9. React Frontend Structure

```
src/
  pages/
  Login.tsx / Register.tsx
  admin/
  employer/
  candidate/
  components/
  services/ # Axios API calls
  context/ # Auth state
  routes/ # Protected routes
  types/ # TypeScript models
```

10. Security & Authorization

Use password hashing rather than storing plain passwords. Issue JWT access tokens at login. Protect backend endpoints with ASP.NET Core [Authorize] and role restrictions. Example: Admin-only endpoints use Admin role; job creation and candidate management use Employer role; job applications and candidate features use Candidate role.

Recommended controls: server-side validation, DTOs, global exception handling, CORS configuration, secure configuration/secrets, file-type and file-size validation for resumes, and parameterized/EF Core queries.

11. Suggested Project Folder Structure

```
JobPortal/
  JobPortal.API/
  Controllers/
  Models/
  DTOs/
  Data/
  Services/
  Middleware/
  Program.cs
  jobportal-client/
  src/
  README.md
```

12. Development Plan — 15 Days

Days	Work
1–2	Create React + ASP.NET Core solution, SQL Server database, EF Core, Git setup
3–4	Register/Login, password hashing, JWT, Admin/Employer/Candidate authorization
5–6	Employer profile, create/edit/close jobs, job approval workflow
7–8	Candidate profile, PDF resume upload, job search and job details
9–10	Applications, application status, employer application dashboard
11–12	Shortlisting, rejection, interview scheduling and tracking
13	AI Resume Analyzer + store analysis in SQL Server
14	Skill match percentage + AI job recommendations
15	Testing, Swagger, validation, UI polish, README and deployment preparation

13. Final End-to-End Flow

1. User registers → 2. Login → 3. JWT issued → 4. Role-based dashboard → 5. Employer creates job → 6. Admin approves → 7. Candidate searches → 8. AI calculates skill match → 9. Candidate applies → 10. Employer reviews → 11. Shortlist → 12. Interview → 13. Selected/Rejected.

AI side flow: Resume PDF → text extraction → OpenAI resume analysis → store structured analysis → compare with job requirements → skill match percentage → recommend suitable approved jobs.

14. Final Project Definition

AI-Powered Job Portal is a full-stack recruitment platform using React for the UI, ASP.NET Core Web API for business logic and REST APIs, SQL Server for persistent data, JWT for authentication, and role-based authorization for Admin, Employer and Candidate. OpenAI is used only where it adds direct value: analyzing resumes, calculating skill matching, and recommending relevant jobs.

This scope keeps the project realistic, demonstrable, and manageable while still showing modern full-stack development and practical AI integration.